VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



CO3133 – Deep Learning and Its Applications

Semester 261

ASSIGNMENT 1 REPORT

Fashion-MNIST

Image Classification

Introduction and Exploratory Data Analysis

Stage 1: Sections 1 and 2 of the complete report

Instructor	Lê Thành Sách
Group member	**Student ID**
Lê Phước Vũ	2353341
Nguyễn Vũ Long	2352696
Nguyễn Hoàng Quốc	2353027

Local reference platform: NVIDIA GeForce GTX 1660 SUPER (6 GiB)

Ho Chi Minh City, 20 October 2026

Abstract / Executive Summary

This assignment studies the classification of low-resolution clothing and accessory images using five neural network configurations: a linear classifier, a multilayer perceptron (MLP), a convolutional neural network (CNN), a row-sequence long short-term memory network (LSTM), and a patch-based Transformer. The central question is how these different input representations and model structures compare when they share the same data partition, preprocessing, and nominal training budget.

This first report stage establishes the problem and the dataset evidence needed for that comparison. Fashion-MNIST provides 60,000 official training images and 10,000 official test images, each with 28 by 28 grayscale pixels and one of ten labels. A seed-42 stratified split produces 54,000 training and 6,000 validation images. Every class contributes 5,400 training, 600 validation, and 1,000 test examples. Descriptive analysis uses the training subset; validation and test data are used here only for structural checks and an exact-duplicate audit.

Across all 42,336,000 training pixels, the raw mean is 72.98 and the population standard deviation is 90.06 on the 0–255 scale. Exactly 50.21% of training pixels are zero, while 0 training images are entirely zero. Class-specific brightness, spatial averages, and variability maps reveal substantial differences in silhouette and image coverage. Similarity between upper-body garment prototypes motivates later class-specific error analysis; these descriptive similarities do not determine a trained model's performance.

The preprocessing pipeline uses a training-only mean of 0.286209 and standard deviation of 0.353160 on the 0–1 scale. The same normalized pixels are exposed as vectors, image grids, row sequences, or patch sequences without discarding information. CUDA reductions were executed on the local GTX 1660 SUPER and checked against CPU calculations. This document completes Sections 1 and 2. Methodology, matched CUDA experiments, model results, and conclusions belong to subsequent stages and are specified in the accompanying continuation guide.

Keywords: Fashion-MNIST; image classification; exploratory data analysis; training-only normalization; CUDA; reproducibility.

Contents

Abstract / Executive Summary i

1 Introduction 1

1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives and Scope . . . 2
1.4 Proposed Approach . . . 2

2 Dataset and Exploratory Analysis 3

2.1 Dataset Description . . . 3
2.2 Data Preparation and Splitting . . . 4
2.3 Exploratory Data Analysis . . . 5
2.4 Preprocessing . . . 9
2.4.1 Tensor Conversion and Intensity Scaling . . . 9
2.4.2 Standardization Using Training Statistics . . . 10
2.4.3 Preventing Leakage Between Partitions . . . 10
2.4.4 Input Representations for the Five Models . . . 11
2.4.5 Data Retention and Augmentation Policy . . . 12
2.4.6 Preprocessing Verification . . . 13
2.5 Key Findings and Implications for Methodology . . . 13

References 15

1 Introduction

1.1 Background and Motivation

Image classification maps an image to a semantic category. In an online product catalogue, for example, an automatic classifier could help organize incoming article images into clothing and accessory categories. The classification itself is only one part of the system: the reliability of the result also depends on the dataset, the representation of each image, the separation between training and evaluation data, and the way model quality and computational cost are measured.

Fashion-MNIST is a useful setting for studying this pipeline. Its compact images allow several neural network configurations to be investigated on the available desktop GPU. At the same time, the recognition problem contains visually similar categories and variation within each category. This combination supports a practical assignment in which the data can be inspected carefully and the experiments can be reproduced. The dataset was introduced by Xiao, Rasul, and Vollgraf as a benchmark based on fashion article images[1].

Different neural architectures organize the same input information differently. A linear classifier combines pixel values directly, while an MLP adds learned nonlinear transformations. A CNN processes neighborhoods on the image grid. An LSTM can read the image as an ordered sequence of rows, and a Transformer can operate on a sequence of small image patches. Comparing these configurations provides a way to investigate how spatial structure, sequence ordering, and model capacity interact with a fixed classification task.

The motivation is therefore both predictive and experimental. A useful study should identify which implemented configuration performs well, describe the classes it confuses, and explain the resources it requires. A common data protocol is essential: otherwise differences in splitting, normalization, checkpoint selection, or hardware could obscure the comparison.

1.2 Problem Statement

The task is supervised, single-label, ten-class image classification. Let $X \in \{0, \dots, 255\}^{28\times 28}$ be a raw grayscale image and $y \in \{0, \dots, 9\}$ its reference label. A classifier with parameters θ receives a preprocessed representation $T(X)$ and produces ten real-valued class scores, or logits:

$$f_\theta(T(X)) = \boldsymbol{z} \in \mathbb{R}^{10}, \qquad \widehat{y} = \underset{k\in\{0,\dots,9\}}{\arg\max}\ z_k. \tag{1}$$

The output is one category for the complete image. Object detection, segmentation, and multilabel classification are outside the scope of this task.

The main visual challenge is that 28 by 28 grayscale images retain only limited detail. A shirt, pullover, coat, and T-shirt/top can have similar overall shapes, while sleeve lengths, openings, and outlines vary within the same label. The EDA in Section 2 examines these properties directly. Such observations motivate later hypotheses about confusion, rather than proving which classes will be hardest for a trained classifier.

The experimental problem is to make the comparison interpretable. A difference in accuracy can reflect input layout, capacity, regularization, optimization, or the training budget. A difference in recorded train-

ing time can also reflect the execution platform. The report must therefore distinguish measured behavior of these particular configurations from general claims about architecture families.

1.3 Objectives and Scope

The study has four objectives:

1. Establish a reproducible Fashion-MNIST data protocol, with a fixed stratified split, training-only normalization, and auditable data provenance.
2. Compare Linear and MLP baselines with the implemented CNN, LSTM, and Transformer, keeping the dataset and nominal optimization budget consistent.
3. Evaluate predictive quality using accuracy, macro-F1, class-specific measures, and confusion matrices, together with learning curves and image-level error examples.
4. Measure parameter count, training cost, inference latency, and GPU memory on a documented reference platform, so that quality and efficiency can be interpreted together.

This first stage completes the introduction and dataset analysis. It verifies the local files, the existing split, class counts, pixel statistics, image variation, exact duplicates, and preprocessing layouts. It does not introduce new model scores or an architecture ranking. The remaining sections will document the implementation, verify experimental artifacts, and interpret the results under the agreed protocol.

The dataset is limited to Fashion-MNIST. No additional product photographs, external training data, pretrained models, or synthetic images are introduced. The shared configuration uses seed 42, ten epochs, batch size 128, Adam with a learning rate of 0.001, and no data augmentation. These are the existing assignment settings, not the outcome of a new hyperparameter search.

The local computational reference is the NVIDIA GeForce GTX 1660 SUPER with 6 GiB of GPU memory, using the existing Windows Python and CUDA PyTorch environment. Existing Linear and MLP training summaries identify MPS as their original training device; those durations cannot serve as a matched CUDA training comparison. A subsequent experimental stage must obtain measurements on this PC under consistent timing scopes and preserve the historical artifacts. Equal epochs also do not imply equal wall-clock computation or equal parameter capacity. The final discussion must retain these qualifications.

1.4 Proposed Approach

The approach begins by separating data inspection from model selection. First, verify the local Fashion-MNIST files and reproduce the cached seed-42 training/validation indices. Next, inspect training images and compute class balance, brightness distributions, nonzero-pixel coverage, spatial class statistics, and exact-duplicate checks. Validation and test images do not contribute to the normalization statistics or training-set visual summaries.

After inspection, scale each raw image to the unit interval and standardize it using one mean and standard deviation estimated from the 54,000 training images. Change only the layout for the selected model: a 784-value vector for Linear and MLP, a one-channel spatial grid for CNN, 28 ordered row steps for LSTM, or 49 ordered 4 by 4 patches for Transformer. Section 2.4 defines these transformations and

checks that all 784 pixel values are retained.

The later training stage will use the shared optimization protocol and choose each model's checkpoint by its lowest validation cross-entropy. The fixed checkpoint will then be evaluated on the official test partition. Results will be interpreted through aggregate metrics, class-specific errors, learning curves, and matched CUDA resource measurements. Additional runs, if performed, will be identified explicitly as repetitions or ablations rather than mixed with the original experiment.

This workflow makes each report claim traceable to code or an exported artifact. In the present stage, the numerical EDA values, figure files, CSV tables, and verification record are generated by `report/generate_eda.py`. The next-stage guide defines the remaining sections and the evidence needed before the report can be described as complete.

2 Dataset and Exploratory Analysis

2.1 Dataset Description

Fashion-MNIST contains fashion article images associated with ten product categories[1, 2]. The local copy has 60,000 official training examples and 10,000 official test examples. Each image is a 28 by 28 array of unsigned 8-bit grayscale intensities; the measured raw range is 0–255. Each record has one integer label. Table 1 gives the label mapping used by the source dataset and the implementation.

Table 1. Fashion-MNIST class labels. Each label is a distinct classification target.

Label	Category	Label	Category
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

The project loads these files through `torchvision.datasets.FashionMNIST`[3]. The analysis script uses the local files with downloading disabled, so a report rebuild does not silently replace the dataset. All four compressed IDX files match the MD5 checksums published by the source repository[2]. SHA-256 digests of the compressed and extracted files, together with their byte sizes, are recorded in `report/data/eda_metrics.json`. The training image and label arrays also reproduce the source fingerprint stored in the project's split cache.

The source repository lists an MIT license[2]. The present report describes the local dataset content rather than assigning an unsupported version number. Its reproducibility anchors are the file digests, array fingerprint, split fingerprint, and the loader/software versions recorded by the generator.

The dataset is appropriate for a controlled assignment, but its images do not cover the full range of real product photography. Color, detailed texture, multiple objects, and unconstrained backgrounds are not represented by this 28 by 28 grayscale task. High accuracy on this benchmark would therefore require separate evidence before being generalized to a production catalogue.

2.2 Data Preparation and Splitting

The official 60,000-image training source is split using `train_test_split` with `stratify=labels`, `test_size=0.1`, and `random_state=42`[4]. The resulting 54,000 training and 6,000 validation indices match the existing `split_cache.json` exactly. The validation fraction is 10% of the official training source; it is not a new split of all 70,000 images. The official 10,000-image test source remains its own partition.

Table 2. Frozen dataset partitions and their experimental roles. Counts are verified locally.

Partition	Images	Per class	Role
Training	54,000	5,400	Fit normalization and update model parameters.
Validation	6,000	600	Select the lowest-validation-loss checkpoint.
Official test	10,000	1,000	Evaluate the fixed selected checkpoint.

The index audit confirms that training and validation are disjoint and that their union covers all 60,000 source indices exactly once. Source and split SHA-256 fingerprints agree with the cache. These checks establish membership and ordering, rather than relying only on the number of examples. Figure 1 summarizes how information passes through the protocol. Training statistics are reused unchanged by validation and test preprocessing.

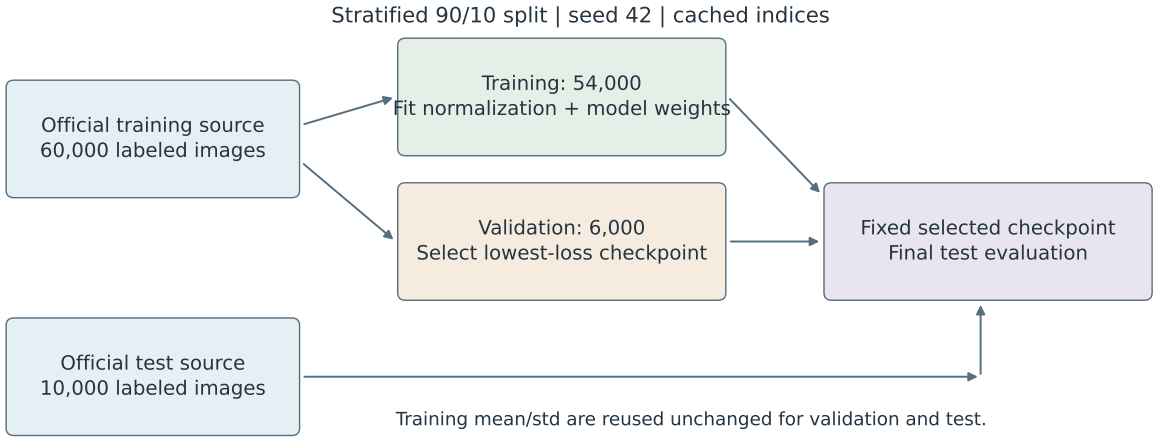


Figure 1. Data flow for the fixed split. Only training images fit normalization and model weights. Validation selects the checkpoint; the selected checkpoint is used for the final official test evaluation.

Exact-duplicate and integrity audit. SHA-256 hashes of the raw 784-byte image arrays identify identical image content independently of its index. Figure 2 reports both within-partition duplicate excess and cross-partition overlap. Duplicate excess is the number of images beyond one representative per distinct hash; cross-partition overlap is the number of distinct hashes appearing in both partitions. These quantities have different denominators and are not interchangeable.

This is an audit of the fixed benchmark, not a reason to tune against the test partition. Descriptive pixel distributions, example selection, and spatial summaries below use training images only. Validation and test are inspected here for dimensions, labels, class counts, file integrity, and exact duplicates. The complete audit includes duplicate-group sizes and conflicting-label-group counts in the JSON evidence file. Exact hashing does not detect near duplicates or related articles photographed separately. Product-level

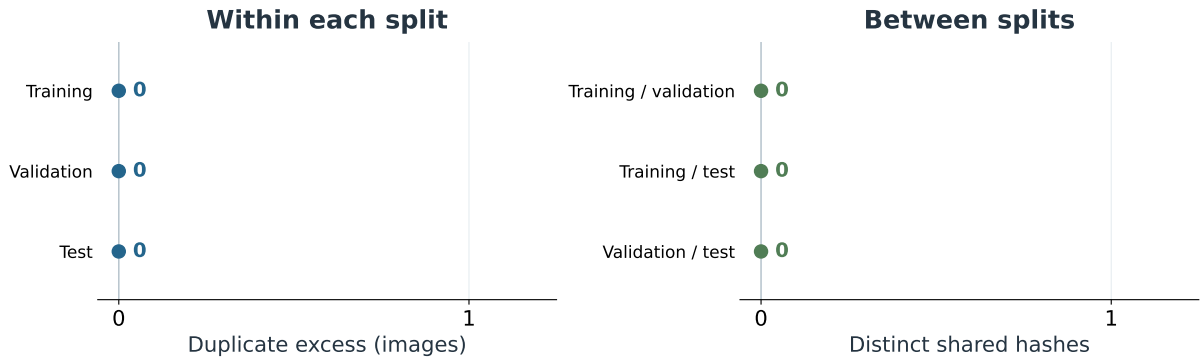


Figure 2. Exact raw-image duplicate audit. All six counts are zero; labeled dots make zero values visible. Left: duplicate excess within each split. Right: distinct hashes shared between pairs of splits. No samples are removed or reassigned.

identifiers are not part of the local image/label arrays, so product-level independence cannot be verified from this check.

2.3 Exploratory Data Analysis

The analysis below uses all 54,000 training images unless an explicitly sampled display is described. Numerical reductions execute on the local CUDA GPU; NumPy checks independently reproduce the intensity histogram and raw moments. Charts display observations or descriptive summaries, not model predictions.

Class balance. Every class accounts for exactly 10% of each partition. The maximum-to-minimum class-count ratio is 1.000 for training, validation, and test (Figure 3). No class weighting, oversampling, or undersampling is introduced. Equal frequencies do not imply equal classification difficulty: class-specific precision, recall, and macro-F1 will still be needed later.

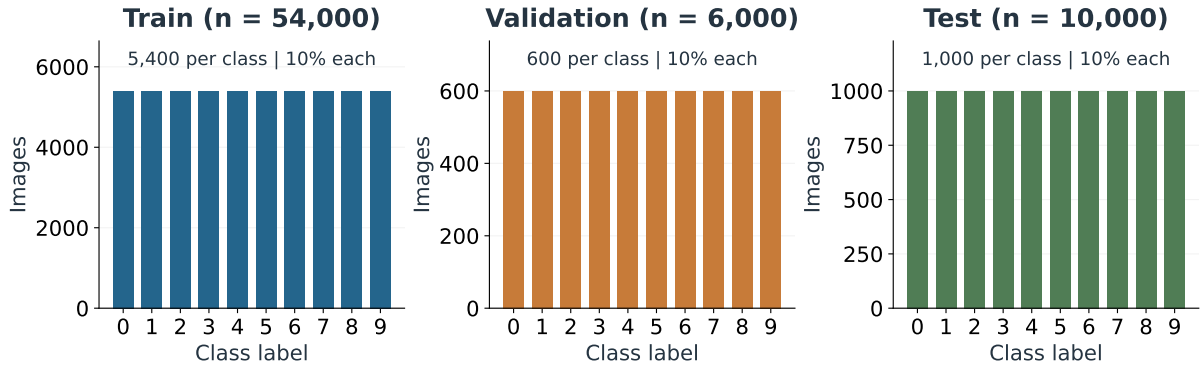


Figure 3. Class counts for each partition. Labels 0–9 follow Table 1. Each panel uses its own count scale, and the exact per-class count is stated above the bars.

Individual image variation. Figure 4 displays three randomly selected training examples per class using NumPy seed 42. Selection is independent of model predictions; the official training-source indices are printed beneath the images and saved in the evidence file. Each image uses the same 0–255 grayscale range, so its displayed brightness is comparable with the other examples.

Several broad silhouettes are visible. Trousers have separated lower legs; bags have compact bodies; and footwear is commonly oriented horizontally. Upper-body garments often share sleeves and a central torso region. Within each class, outline and brightness vary. These examples provide context for the quantitative summaries, but three examples per class cannot establish the frequency of a visual pattern.

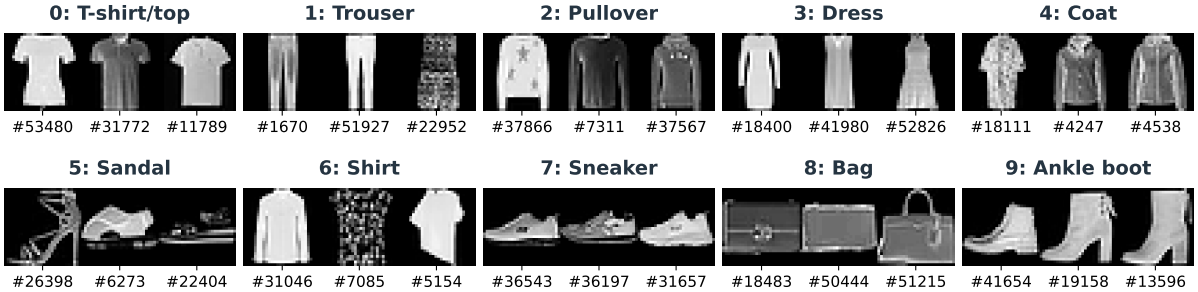


Figure 4. Three seed-42 training examples for each category, with original source indices. The display preserves each image's 28 by 28 pixels and uses nearest-neighbor rendering with a common intensity scale.

Pixel intensity and background. The training subset contains $54{,}000 \times 28 \times 28 = 42{,}336{,}000$ pixels. On the raw scale, the pixel-weighted mean is 72.98 and the population standard deviation is 90.06. Exactly 50.21% of pixels have value zero. Nevertheless, there are 0 entirely zero training images: a large zero-pixel mass does not mean that the observations are blank.

Figure 5 uses two distributions to make this structure readable. The left panel includes every pixel and shows the large mass at zero. The right panel conditions on positive intensity values, with its own denominator, so that variation among nonzero pixels is visible. The zero mass is compatible with dark background regions, but a zero/nonzero test is not a semantic foreground segmentation.

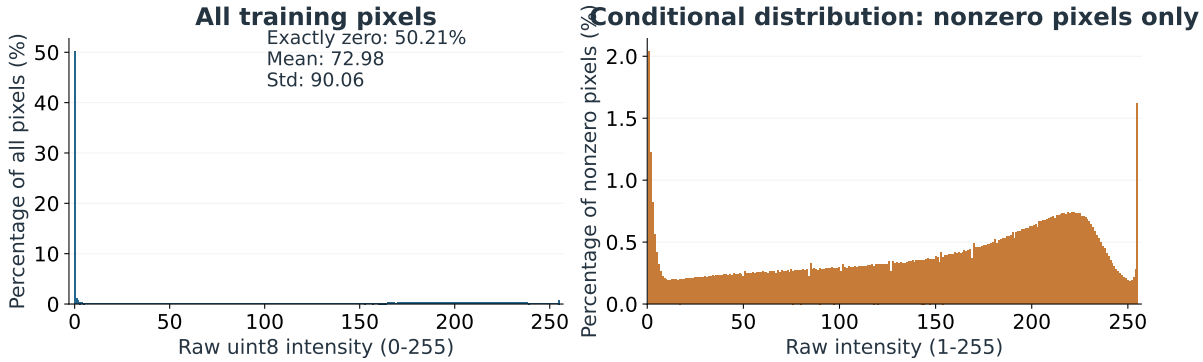


Figure 5. Training-pixel intensity distributions. The left panel is normalized by all 42,336,000 training pixels; the right panel is normalized by nonzero pixels only. Percentages therefore refer to different populations. No validation or test pixels enter these distributions.

Brightness and nonzero coverage by class. For image i, define its average raw brightness b_i and nonzero-pixel coverage c_i by

$$b_i = \frac{1}{784} \sum_{r=0}^{27} \sum_{c=0}^{27} X_{i,r,c}, \qquad c_i = \frac{1}{784} \sum_{r=0}^{27} \sum_{c=0}^{27} \mathbb{1}[X_{i,r,c} > 0]. \tag{2}$$

Coverage counts every positive value, including faint edge pixels. It is a descriptive statistic with a fixed threshold of zero, not an estimated object area or a segmentation annotation.

Figure 6 shows medians, interquartile ranges, and whiskers for both quantities. Table 3 reports class means and the population standard deviation of image-level brightness. Sandal has the lowest mean brightness (34.87) and Coat has the highest (98.15). These are group averages; the distributions overlap, so brightness alone is not established as a sufficient classification feature.



Figure 6. Image-level brightness and nonzero coverage across the ten training classes. Boxes show the middle 50%, black lines show medians, and whiskers extend to observations within 1.5 interquartile ranges of the box. Outlier markers are hidden for readability; all observations remain in the calculations.

Table 3. Training-set class statistics. Each class has 5,400 images. Brightness SD is across image means, not across individual pixels.

Label	Class	Mean brightness	Brightness SD	Mean nonzero (%)
0	T-shirt/top	83.02	28.84	59.38
1	Trouser	56.91	14.55	34.91
2	Pullover	96.17	32.70	64.84
3	Dress	65.91	21.33	42.67
4	Coat	98.15	26.46	60.19
5	Sandal	34.87	15.74	32.10
6	Shirt	84.84	32.90	62.87
7	Sneaker	42.81	12.85	33.81
8	Bag	90.28	28.81	58.61
9	Ankle boot	76.87	18.32	48.48

Spatial class averages and variability. For each class k, let $\mathcal{I}_k$ contain its training images after scaling to $[0, 1]$. The pixelwise prototype M_k and population variability map S_k are

$$M_{k,r,c} = \frac{1}{n_k} \sum_{i \in \mathcal{I}_k} U_{i,r,c}, \qquad S_{k,r,c} = \sqrt{\frac{1}{n_k} \sum_{i \in \mathcal{I}_k} (U_{i,r,c} - M_{k,r,c})^2}, \quad U_i = X_i/255. \tag{3}$$

Figure 7 separates these two summaries. Mean maps show recurring silhouettes at their original image positions. Standard-deviation maps show where intensity changes across examples of the same class. Variable outlines, alignment, and material appearance can all contribute; the maps do not isolate these causes individually.

The common scale within each map type is deliberate. Independently rescaling each prototype could conceal brightness differences between classes. The mean maps use 0–1 and the standard-deviation maps use 0–0.5, the maximum population standard deviation for a variable bounded by 0 and 1. A prototype is an average, not an actual article image, and blur in an average can reflect variation rather than low quality in every observation.

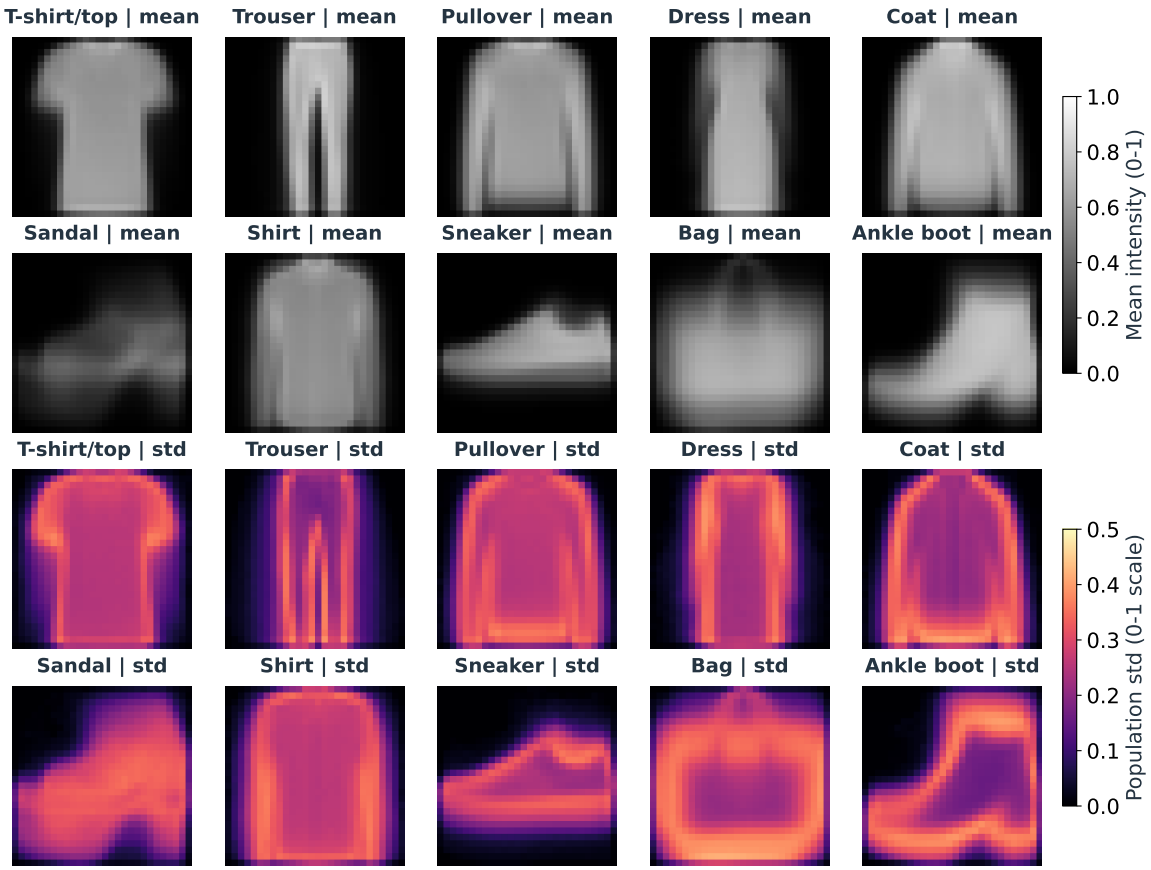


Figure 7. Class-average images (top two rows) and population standard-deviation maps (bottom two rows), each computed from all 5,400 training images of its class. Shared color scales allow comparisons within each map type.

Similarity of class prototypes. To describe similarities visible in the mean maps, flatten each class prototype into $\boldsymbol{m}_k \in \mathbb{R}^{784}$ and calculate

$$s_{k\ell} = \frac{\boldsymbol{m}_k^\top \boldsymbol{m}_\ell}{\|\boldsymbol{m}_k\|_2 \|\boldsymbol{m}_\ell\|_2}. \tag{4}$$

The vectors are uncentered, nonnegative training class means, so the cosine similarities lie between zero and one. Figure 8 gives the full matrix. The largest off-diagonal value is 0.9873 for Coat and Shirt. The reported pair is selected from all distinct class pairs; the diagonal simply compares a class with itself.

This statistic summarizes average spatial appearance. It ignores individual image variation and is sensi-

tive to alignment and broad foreground layout. It is therefore neither a confusion matrix nor an estimate of the achievable classification error. Its role is to propose a question for later error analysis: do highly similar garment prototypes correspond to common mistakes in the evaluated models?

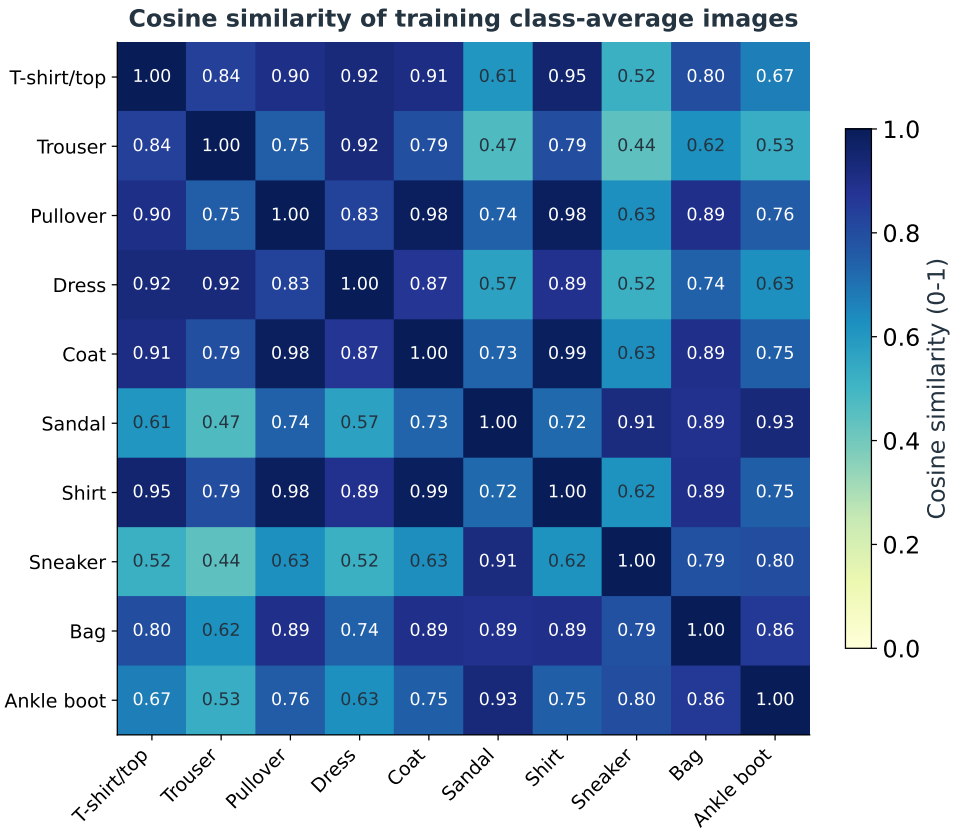


Figure 8. Cosine similarity between flattened training class-average images. Values are rounded to two decimal places for display. The unrounded maximum off-diagonal similarity is reported in the text; this chart contains no model predictions.

2.4 Preprocessing

The EDA shows that the images already share a fixed resolution and class mapping, but differ in brightness, coverage, and spatial appearance. Our preprocessing therefore keeps the original image content and applies a common numerical transformation before arranging the pixels for each model. The pipeline has four steps: convert the raw image to a floating-point tensor, scale its intensities, standardize it using training statistics, and construct the required input layout. The same fitted transformation is used for training, validation, and test images.

2.4.1 Tensor Conversion and Intensity Scaling

Let $X_i \in \{0, \ldots, 255\}^{28\times28}$ be a raw image. The loader uses `ToTensor` to add a channel dimension, convert the unsigned 8-bit values to `float32`, and divide the intensities by 255:

$$U_{i,0,r,c} = \frac{X_{i,r,c}}{255}, \qquad U_i \in [0,1]^{1\times28\times28}. \tag{5}$$

Here, $r, c \in \{0, \ldots, 27\}$ denote the row and column, and the channel index is zero because the image is grayscale. For example, raw intensities 0, 128, and 255 become approximately 0, 0.5020, and 1. This conversion changes the numeric range while preserving the 784 pixels, their order, and the supplied label. Native resolution is retained; no interpolation or resizing is needed. Scaling takes place on the CPU in the dataset loader, before a batch is transferred to the selected training device.

2.4.2 Standardization Using Training Statistics

Scaling to $[0, 1]$ and standardization serve different purposes. Scaling sets a common numerical range; standardization centers that range relative to the training distribution and expresses variation in units of its standard deviation. We use one mean and one population standard deviation for the single grayscale channel, computed across all training images and positions. With $\mathcal{I}_{\text{tr}}$ denoting the frozen set of 54,000 training indices and $N = 54{,}000 \times 784 = 42{,}336{,}000$, the statistics are

$$\mu = \frac{1}{N} \sum_{i \in \mathcal{I}_{\text{tr}}} \sum_{r=0}^{27} \sum_{c=0}^{27} U_{i,0,r,c}, \qquad \sigma = \sqrt{\frac{1}{N} \sum_{i \in \mathcal{I}_{\text{tr}}} \sum_{r=0}^{27} \sum_{c=0}^{27} (U_{i,0,r,c} - \mu)^2}. \tag{6}$$

The measured values are $\mu = 0.286209$ and $\sigma = 0.353160$, rounded here to six decimal places. The implementation retains the full precision stored in the split cache. Each scaled image is transformed as

$$Z_{i,0,r,c} = \frac{U_{i,0,r,c} - \mu}{\sigma}. \tag{7}$$

These are global training-pixel statistics, rather than separate statistics for each image, class, or batch. Applying the same affine transformation to every image preserves differences in brightness between examples. Computing separate statistics for each class would also require the unknown target label when preprocessing a new image, so it is not part of this pipeline.

Table 4 summarizes the three numerical stages. After standardization, raw zero-valued background pixels become approximately -0.810, and raw values of 255 become approximately 2.021. Negative values are expected and are retained. The transformation is invertible up to floating-point rounding: $U = \sigma Z + \mu$. It does not make the image distribution Gaussian or remove the background.

Table 4. Numerical scales across all training pixels. Values are rounded; standardization is global, not performed separately for each image.

Representation	Mean	Population SD	Minimum	Maximum
Raw intensity	72.983297	90.055735	0	255
Scaled intensity	0.286209	0.353160	0	1
Standardized intensity	≈ 0	≈ 1	-0.810424	2.021156

2.4.3 Preventing Leakage Between Partitions

The split described in Section 2.2 is fixed before fitting the normalization. Only its 54,000 training images contribute to μ and σ. The validation and test transforms reuse those constants without recalculation; their own normalized distributions are not required to have exactly zero mean or unit standard deviation. Fitting statistics on all 70,000 images, or normalizing each evaluation partition using its own statistics,

would allow held-out information to influence the input transformation.

The cached training and validation indices, normalization constants, source fingerprint, and split fingerprint identify the preprocessing used by a run. All five architectures use this same split and fitted numerical transform. Validation loss selects the checkpoint, while the official test set evaluates the selected weights. Neither test labels nor test performance determine the normalization or the choice of representation in the baseline comparison.

2.4.4 Input Representations for the Five Models

After standardization, the representation utility arranges the same 784 values for each architecture. Table 5 gives the shapes after batching, where B is the number of images in a batch. The normal batch size is 128; the final batch may contain fewer images. Inputs remain `float32`. Each image keeps one integer target in $\{0, \ldots, 9\}$, giving a target tensor of shape $(B,)$. Labels are not standardized or converted into image features.

Table 5. Model input layouts after common standardization. Shapes precede learned feature extraction or patch embedding. Each image retains 784 values.

Model	Batch shape	Ordering and interpretation
Linear / MLP	$(B, 784)$	One vector per image, flattened row by row.
CNN	$(B, 1, 28, 28)$	One grayscale channel with the original image grid.
LSTM	$(B, 28, 28)$	28 row steps, with 28 pixel features at each step.
Transformer	$(B, 49, 16)$	49 nonoverlapping 4×4 patches, with 16 pixel features per patch.

Linear Classifier and MLP. The image is flattened in row-major order:

$$v_{i,28r+c} = Z_{i,0,r,c}, \qquad v_i \in \mathbb{R}^{784}. \tag{8}$$

Thus, positions 0–27 contain the first row and positions 28–55 contain the second. Flattening preserves the values and their fixed positions, but the model interface does not explicitly encode a two-dimensional neighborhood or impose local weight sharing. The MLP's hidden transformations belong to the model architecture, rather than to preprocessing.

Convolutional Neural Network. The CNN receives Z_i directly as a $(1, 28, 28)$ image. Keeping the channel and spatial axes allows convolutional layers to operate on local neighborhoods. No handcrafted edge map, threshold mask, or class prototype is supplied as an additional channel.

LSTM. The single channel dimension is removed, giving

$$S_{i,t,j} = Z_{i,0,t,j}, \qquad t, j \in \{0, \ldots, 27\}. \tag{9}$$

The sequence moves from the top row to the bottom row, and each step contains the 28 left-to-right pixel values in that row. These are spatial steps within one static image. Because both axes have length 28, an accidental transpose would preserve the shape while changing the sequence; ordering must therefore be checked by comparing the actual elements.

Transformer. The image is divided into a 7×7 grid of nonoverlapping 4×4 patches. Patches are read

left to right and then top to bottom, and the pixels inside each patch are also flattened row by row. If $a, b \in \{0, \ldots, 6\}$ index the patch grid and $u, v \in \{0, \ldots, 3\}$ index a pixel within a patch, then

$$P_{i,7a+b,\,4u+v} = Z_{i,0,\,4a+u,\,4b+v}. \tag{10}$$

There are 49 patches with 16 values each, so $49 \times 16 = 784$. The patch layout requires no padding or discarded boundary pixels. Projection from 16 pixel values to an embedding, positional encoding, and attention are model operations to be explained in the methodology section.

Figure 9 illustrates the layouts using one training image. Reordering preserves the pixel information; differences in how the architectures learn from these layouts will be examined experimentally.

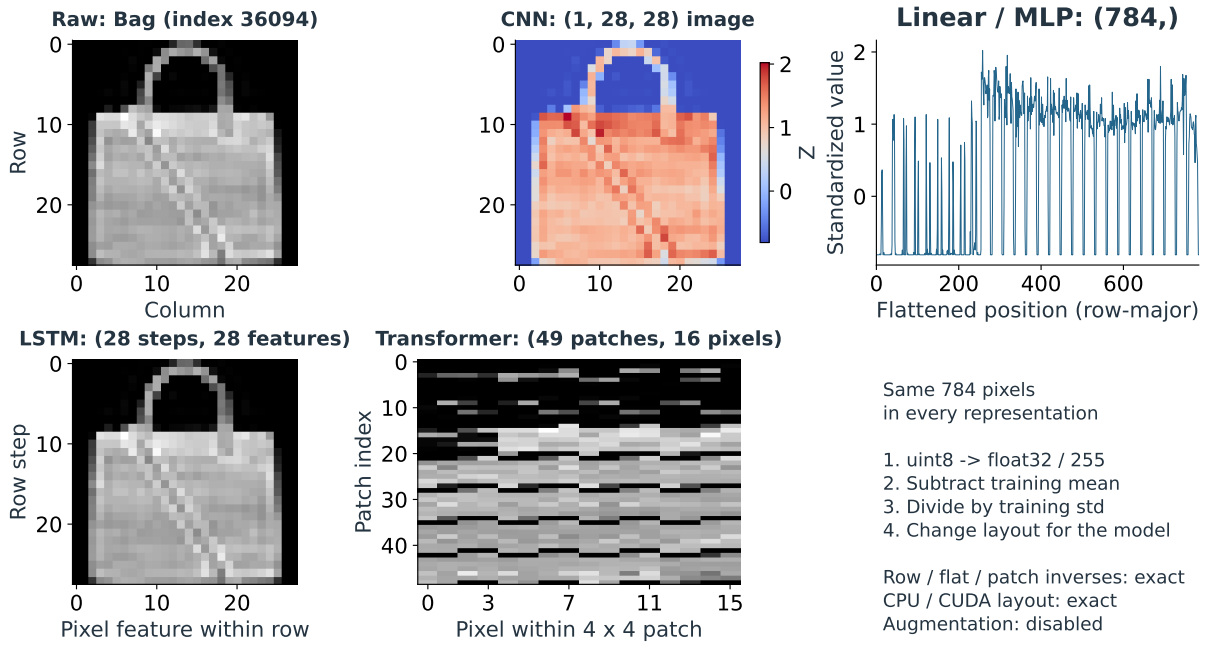


Figure 9. A training image shown as raw pixels and as standardized inputs for the five models. Vector, row, and patch layouts contain the same 784 standardized values; only their arrangement changes. The example is not a model prediction.

2.4.5 Data Retention and Augmentation Policy

The baseline retains every image and all pixel coordinates. The audit found no entirely blank training images or exact duplicates across the checked partitions. Brightness and nonzero coverage vary within each class, but extreme values alone do not establish corruption or an incorrect label. Accordingly, the EDA does not justify removing dark images, cropping stable background locations, or relabeling visually ambiguous garments.

No resizing, denoising, binarization, histogram equalization, or dimensionality reduction is applied to the model inputs. In particular, the nonzero masks, class means, and similarity matrix in the EDA remain descriptive analyses; they are not extra training features or replacements for the original images. All five models therefore start from the same grayscale information.

Random augmentation is disabled in the shared baseline configuration. This keeps the initial comparison independent of architecture-specific augmentation choices. The loader supports optional horizontal flips

with probability 0.5 and rotations of up to ten degrees for training images, but these options are not part of the reported preprocessing baseline. If studied later, they must be identified as separate experiments, applied only to training inputs before standardization, and evaluated with the same deterministic validation and test transforms. No augmentation gain is assumed here.

2.4.6 Preprocessing Verification

The saved EDA audit checks the transformation against the existing data utility. Over all standardized training pixels, the measured mean is within 10^{-6} of zero and the population standard deviation is within 10^{-6} of one. Independent NumPy calculations reproduce the raw moments and pixel histogram, and the fitted normalization agrees with the split cache. The same CPU tensor conversion is used by the loader and the EDA before CUDA calculations, avoiding a change in conversion order between them.

Representation checks compare flattened and row-sequence elements with their source pixels, reconstruct the image from the patch sequence, and confirm agreement between CPU and CUDA layout conversions. These checks verify values and ordering, rather than relying only on tensor dimensions. Together with the fixed split, they establish the input interface for the methodology and implementation sections. They do not establish classifier accuracy, which requires evaluation of trained models.

2.5 Key Findings and Implications for Methodology

The dataset analysis establishes a controlled starting point for the five-model comparison. Its main findings translate into the following experimental decisions.

1. **Class balance supports an unweighted baseline.** Each class contributes 5,400 training, 600 validation, and 1,000 test images. No resampling or class weighting is introduced to address frequency imbalance. Equal counts do not guarantee equal recognition difficulty, so evaluation must include per-class precision, recall, and F1 alongside accuracy and macro-F1.

2. **Background prevalence must be separated from image validity.** Exactly 50.21% of training pixels are zero, but no training image is entirely blank. Brightness and coverage distributions overlap across classes. The baseline therefore keeps the grayscale intensities and every image rather than applying an unsupported brightness threshold or background removal rule.

3. **Spatial structure motivates different model representations.** Class means retain broad silhouettes, while variability maps and individual examples show changes that averaging conceals. Vectors provide the Linear and MLP baselines; the CNN retains the image grid; the LSTM reads ordered rows; and the Transformer receives ordered patches. All layouts retain the same 784 standardized values. Architecture comparisons must still account for differences in parameter count and computation before attributing a performance gap to representation alone.

4. **Prototype similarity gives a specific error-analysis question.** Coat and Shirt have the highest off-diagonal prototype cosine similarity, approximately 0.9873. Later confusion matrices should check whether these and other upper-body categories are frequent directional errors, and example galleries should examine the corresponding mistakes. Similarity of averages alone does not predict which model will perform best or establish that any supplied label is incorrect.

5. **One fitted preprocessing pipeline supports a consistent comparison.** The seed-42 split, cached

indices, training-only normalization, and verified input ordering are shared by all architectures. Validation loss selects weights, and the selected checkpoint is evaluated on the official test set. Any change to augmentation or representation must be recorded as a separate experimental condition rather than silently changing the baseline.

The analysis has limits. Exact-image hashing cannot exclude near duplicates or related products, and a few sampled images cannot establish the frequency of an appearance. Class averages hide individual variation. The present evidence also does not measure training variability or generalization to high-resolution, colored, or unconstrained product photographs. Those limits remain relevant when interpreting later results.

Section 3 can now define the five architectures, their mathematical operations, loss, training strategy, and evaluation metrics using the input shapes established in Section 2.4. Section 4 can describe how the shared dataset views, batching, random-state control, device transfer, and checkpoint handling implement that protocol. Training histories, predictive results, timing measurements, and error analysis belong to the later experimental sections; the EDA provides hypotheses and preprocessing decisions for those experiments rather than substituting for them.

References

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," *arXiv preprint arXiv:1708.07747*, 2017. https://arxiv.org/abs/1708.07747.

[2] Zalando Research, "Fashion-MNIST: dataset repository, label mapping, file checksums, and license." https://github.com/zalandoresearch/fashion-mnist. Accessed 20 October 2026.

[3] PyTorch Contributors, "FashionMNIST," *torchvision 0.24 documentation*. https://docs.pytorch.org/vision/0.24/generated/torchvision.datasets.FashionMNIST.html. Accessed 20 October 2026.

[4] Scikit-learn Developers, "train_test_split," *scikit-learn documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html. Accessed 20 October 2026.